DATA ANALYTICS

DATA MANAGEMENT, DATA VIZ & TEXT MINING

Diplôme Sorbonne Data Analytics

Introduction au cours

Ce module a pour objectif de fournir les bases pratiques et théoriques nécessaires à la manipulation, l'analyse et la visualisation de données à l'aide de Python.

Vous apprendrez à :

- Maîtriser les fondamentaux de Python pour l'analyse et la manipulation de données.
- Gérer et transformer des données efficacement avec Pandas.
- Créer des visualisations claires et pertinentes.
- Analyser des textes (Text Mining) à travers des outils simples.
- Développer des dashboards interactifs pour restituer vos résultats.

Python est le langage central de ce cours : il est incontournable en Data Analytics, Business Intelligence et Data Science.

PLAN DU COURS

1. Remise à niveau en Python
2. Data Management avec Pandas
3. Introduction au Text Mining
4. Visualisation des données
5. Création de dashboard

Remise à niveau en Python

Introduction

Python est un langage de programmation **interprété** et **orienté objet**, largement utilisé en data science, développement web et automatisation.

Voici 4 atouts majeurs de Python :

- **Qualité :** un langage propre, lisible, et orienté objet pour un code évolutif et maintenable.
- **Productivité :** permet d'écrire rapidement du code clair, en déléguant les détails techniques à l'interpréteur.
- **Portabilité :** fonctionne sur toutes les plateformes (du smartphone au serveur).
- **Intégration :** s'interface facilement avec d'autres langages comme C, C++ ou Java.

Python est devenu un outil incontournable en Data Analytics et en Data Science.

Caractéristiques techniques de Python

Python est un langage :

- Interprété : pas de compilation préalable, le code s'exécute ligne par ligne.
- Sans typage explicite : les types sont définis automatiquement lors de l'affectation (x = 5).
- Avec gestion automatique de la mémoire : grâce au comptage de références.
- Multi-paradigme : prend en charge la programmation :orientée objet, procédurale, fonctionnelle.
- Dynamique et interactif : idéal pour le prototypage rapide et le travail en console.
- Capable de générer du byte-code : améliore les performances à l'exécution.
- Facile à interfacer : interaction avec les systèmes, protocoles réseau, fichiers, etc.
- Extensible : s'intègre facilement à des composants en C, C++ ou Java (via Jython).

Ces caractéristiques font de Python un langage polyvalent et accessible, adapté à de nombreux domaines, y compris la manipulation et l'analyse de données.

Les opérateurs mathématiques :

Opérateur	Signification	Exemple	Résultat
+	Addition / Concaténation	2 + 3 / "a" + "b"	5 / "ab"
-	Soustraction	7 - 4	3
*	Multiplication	3 * 2	6
/	Division (résultat float)	5 / 2	2.5
//	Division entière	5 // 2	2
%	Modulo (reste de division)	5 % 2	1
**	Puissance	2 ** 3	8

Les entrées Sorties (Lecture /Ecriture) :

La fonction **print()** permet d'afficher du texte ou des variables à l'écran.

```
print("Bonjour")                    # Bonjour
print("Nom :", "Alice", 25)         # Nom : Alice 25
print("Python", "3", "10", sep="-")  # Python-3-10
print("Ligne 1", end=" | ")
print("Ligne 2")                    # Ligne 1 | Ligne 2

a=10
print(a)                            #10
```

- sep : définit le séparateur entre les éléments.
- end : définit le caractère de la fin d'impression (par défaut, retour à la ligne).

Les entrées Sorties (Lecture /Ecriture) :

La fonction **input()** permet de lire une entrée utilisateur.

- input() lit une chaîne de caractères depuis le clavier.
- On peut laisser les parenthèses vides ou y insérer un message explicatif.
- input() permet de suspendre l'exécution du programme en attendant que l'utilisateur entre une valeur. L'exécution reprend une fois la touche Entrée pressée.
- input() retourne toujours une chaîne (str), donc conversion nécessaire avec int() ou float() si besoin.
- En cas de mauvaise saisie, une erreur peut apparaître (voir gestion d'erreur avec try/except).

```python
prenom = input("Entrez votre prénom : ")
print("Bonjour,", prenom)
print("Veuillez entrer un nombre positif quelconque : ", end=" ")
ch = input()
nn = int(ch) # conversion de la chaîne en un nombre entier
print("Le carré de", nn, "vaut", nn**2)
```

```
Entrez votre prénom : Nisrine
Bonjour, Nisrine
Veuillez entrer un nombre positif quelconque :
2
```

Les variables

Une variable est un nom qui fait référence à une valeur stockée en mémoire.

Aucune déclaration de type n'est nécessaire en Python.

```
nom = "Alice"
age = 30
pi = 3.14
est_actif = True
```

Fonctions utiles :

```
type(x)     # renvoie le type de x
del x       # supprime la variable x de la mémoire
```

Les variables

Les principaux types de variables :

- str → chaîne de caractères
- int → entier
- float → nombre à virgule
- bool → valeur booléenne (True ou False)

Les mots réservés :

Les mots réservés sont des mots-clés prédéfinis que Python utilise pour sa syntaxe.
Ils ne peuvent pas être utilisés comme noms de variables ou de fonctions.

Exemples : if, else, while, for, def, return, True, None, class, etc.

Pour afficher la liste complète dans Python :

```
import keyword
print(keyword.kwlist)
```

Les tests

- **If else & If elif else**

Syntaxe 1

```
if condition1:
     Action1 si condition vraie
     Action2 si condition vraie
     ……….....
else:
     Action1 si condition fausse
     Action1 si condition fausse
     ……….....
```

Syntaxe 2

```
if condition1:
     Action1 si condition1 vraie
     Action2 si condition1 vraie
     ……….....
elif condition2: # si condition1 fausse
     Action1 si condition2 vraie
     Action1 si condition2 vraie
     ……….....
else:
     Action1 si condition2 fausse
     Action1 si cond
```

Les tests

- **Les opérateurs de comparaison (opérateurs logiques)**
  - x **==** y : x est égal à y
  - x **!=** y : x est différent de y
  - x **>** y : x est plus grand que y
  - x **<** y : x est plus petit que y
  - x **>=** y : x est plus grand que, ou égal à y
  - x **<=** y : x est plus petit que, ou égal à y
  - x **is** y : x et y représentent le même objet
  - x **is not** y : x et y ne représentent pas le même objet
  - x **in** y : x est un élément de y (x dans y)
  - x **not in** y : x est non un élément de y
- **Autres opérateurs logiques :**
  - Condition1 **and** condition2
  - Condition1 **or** condition2
  - **not** condition2

Les boucles

- **La boucle while :**

  L'instruction **while** ("tant que", en français) permet d'exécuter une boucle tant qu'une condition est vraie.

```
while Variable différente d'une valeur finale (VF) : # tant que la condition est vraie
     Instruction1 # exécution de l'instruction 1
     Instruction2 # exécution de l'instruction 2
     ……................. #exécution de l'instruction n
     Incrémentation/décrémentation # variable= ± pas (recherche de condition)
```

- **La boucle for :**

  La boucle for permet d'exécuter des instructions définies sur tous les éléments d'une liste ou d'une séquence.

```
for Variable in range(VI,VF, pas) :
     Instruction1 # exécution de l'instruction 1
     Instruction2 # exécution de l'instruction 2
     …................... # exécution de l'instruction n
```

```
for Variable in [v1,v2,v3] :
     Instruction1 # exécution de l'instruction 1
     Instruction2 # exécution de l'instruction 2
     …................... # exécution de l'instruction n
```

Liste en compréhension (List comprehension)

La liste en compréhension offre une syntaxe plus courte lorsque vous souhaitez créer une nouvelle liste basée sur les valeurs d'une liste existante.
Exemple : À partir des nombres de 0 à 9, vous souhaitez créer une nouvelle liste contenant les carrés des nombres pairs uniquement.
Sans compréhension de liste, vous devrez écrire une boucle for contenant un test conditionnel

```
carrés_pairs = []
for x in range(10):
    if x % 2 == 0:
        carrés_pairs.append(x**2)
```

Avec une compréhension de liste, cela devient plus concis :

```
carrés_pairs = [x**2 for x in range(10) if x % 2 == 0]
```

Syntaxe générale :

[expression for élément in iterable if condition]

➔ Plus concis et lisible qu'une boucle for traditionnelle

Fonctions de programmation fonctionnelle

Ce sont des outils issus du paradigme fonctionnel, un style de programmation qui met l'accent sur :

- les fonctions comme objets (on peut les passer en argument)
- l'absence de boucle explicite
- l'usage de fonctions sans effets de bord (ex : lambda) (fonction ne fait que retourner un résultat, elle ne modifie rien autour d'elle)
- le traitement de listes ou itérables de manière déclarative (ex : map(), filter

lambda → fonction anonyme ou fonction lambda
map() → fonction d'application (appliquer une fonction à chaque élément)
filter() → fonction de filtrage

Fonctions de programmation fonctionnelle

lambda → fonction anonyme ou fonction lambda
fonction anonyme, c'est-à-dire sans nom, souvent utilisée pour écrire des fonctions courtes d'une seule ligne.

Exemple :
On veut une fonction qui retourne le carré d'un nombre.

Avec une fonction classique :

```
def carre(x):
    return x**2
```

Avec une fonction lambda :

```
carre = lambda x: x**2
```

Fonctions de programmation fonctionnelle

Les fonctions lambda sont souvent utilisées avec des fonctions telles que **map()** et **filter()**, qui prennent d'autres fonctions comme arguments pour traiter les éléments d'une collection.

Avec une fonction filter() : fonction lambda pour vérifier si un nombre est pair. La fonction filter applique cette fonction lambda à l'ensemble numbers

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
evens = filter(lambda x: x % 2 == 0, numbers)
print(list(evens))                    # [2, 4, 6, 8]
```

Avec une fonction map() : calcul de la longueur des chaînes de caractères d'une liste en associant la fonction len() à chaque élément.

```
fruits = ['apple', 'banana', 'cherry']
lengths = list(map(lambda x: len(x), fruits))
print(lengths)                        #[5, 6, 6]
```

Les instructions « break » et « continue »

- **Break :**

Il est possible de sortir d'une boucle avec l'instruction break. Cette instruction est très pratique pour tester une condition d'arrêt qui dépend d'une valeur entrée. Par exemple:

```
somme = 0
while True:
    n = int(input("Entrez un nombre (0 pour arrêter): "))
    if n == 0:
        break
    somme = somme + n
print("La somme des nombres est", somme)
```

```
Entrez un nombre (0 pour arrêter): 8
Entrez un nombre (0 pour arrêter): 4
Entrez un nombre (0 pour arrêter): 0
La somme des nombres est 12
```

Les instructions « break » et « continue »

- **Continue :**

La fonction continue : retourne directement au début de la boucle, ce qui permet d'éviter des tests de condition inutiles

```
nb=1;
while nb!=0:
    nb=int(input("Entrer un entier "))
    if nb==3:
        print(" choix N° 1 ")
        continue
    print(" valeur lue =",nb)
    print(" Veuillez entrer une autre valeur.")
```

```
Entrer un entier 5
 valeur lue = 5
 Veuillez entrer une autre valeur.
Entrer un entier 1
 valeur lue = 1
 Veuillez entrer une autre valeur.
Entrer un entier 3
 choix N° 1

Entrer un entier
```

L'instruction « pass » :

Il faut voir l'instruction pass comme un commentaire à la différence que lui est interprété par Python.

« pass » a une valeur nulle et sert à éviter les erreurs lors de l'implémentation de boucles, fonctions ou méthodes car ces structures doivent contenir au minimum une instruction.

```
Alphabet = ['A', 'B', 'C', 'D', 'E']

for lettre in Alphabet:
    pass # Je ne sais encore quoi faire avec ces lettres mais je veux tester la suite de mon code

print("Suite du code...")
```

Exercice 1:

Ecrire un programme en python qui affiche les 20 premiers multiples d'un entier entré par l'utilisateur :

1. En utilisant la boucle while
2. En utilisant la boucle for

Les fonctions en python

Une fonction est un sous programme qui réalise une certaine tâche.

Une fonction est composée de trois grandes parties :

- Son **nom** qui permet d'y faire appel et l'identifier des autres fonctions.
- Ses **arguments** qui permettent de spécifier des données à lui transmettre.
- Sa **sortie**, c'est-à-dire ce qu'elle retourne comme résultat.

- Les ***fonctions*** qui effectuent des instructions et retournent un résultat.
- Les ***procédures*** qui effectuent des instructions mais ne retournent rien du tout.

Exemple:

```
def puissance(valeur,puissance)
    res = valeur**puissance
    return res
```

```
def puissance(valeur,puissance)
    res = valeur**puissance
    print(res)
```

Les fonctions en python

- Les fonctions avec des arguments ayant des valeurs par défaut

Exemple:

```
def puissance(valeur,puissance=2):
    res = valeur**puissance
    print(res)

puissance(2,4)
puissance(2)
```

```
16
4
```

```
def puissance(valeur, puissance):
    res = valeur**puissance
    print(res)

puissance(puissance=4,valeur=2)
```

```
16
```

- Les fonctions avec un nombre d'arguments indéfini :
  - Syntaxe 1 avec une séquence : def fonction**(*arguments**)
  - Syntaxe 2 avec un dictionnaire de clé,valeur : def fonction**(**arguments**)

```
def fonction_syn1(*arguments):
    for arg in arguments:
        print(arg)
```

```
def fonction_syn2(**arguments):
    for cle in arguments:
        print(cle,arguments[cle])
```

Les structures de bases

- **Les listes :**

  Une liste est une structure de données de types différents

  - Création d'une liste :
    - L = list() ou L = [] ou L = 'Bonjour madame'.split()

```
L = 'Bonjour madame'.split()
print(L)
```

```
['Bonjour', 'madame']
```

```
L = 'Bonjour madame'.split('m')
print(L)
```

```
['Bonjour ', 'ada', 'e']
```

  - Pour créer une liste d'entiers :
    - L = range(4) , L = range(4,10) , L = range(4,10,2)

```
L = range(4)
print(L)
print(list(L))
```

```
range(0, 4)
[0, 1, 2, 3]
```

```
L = range(4,10)
print(L)
print(list(L))
```

```
range(4, 10)
[4, 5, 6, 7, 8, 9]
```

```
L = range(4,10,2)
print(L)
print(list(L))
```

```
range(4, 10, 2)
[4, 6, 8]
```

Les structures de bases

- **Les listes :**
  - Taille d'une liste : Taille = len(L)

```
L = range(4,10,2)
print(len(L))
```

```
3
```

  - Accès au premier et dernier élement de la liste :
    - Premier = L[0] , dernier = L[-1]
  - Fonctions utiles d'une liste :
    - L.append(x) : ajoute un élément à la fin.
    - L.insert(i, x) : insère x à l'index i.
    - L.remove(x) : supprime la première occurrence de x.
    - L.pop([i]) : supprime et retourne l'élément à l'index i (dernier si non spécifié).
    - L.count(x) : compte le nombre d'occurrences de x.
    - L.clear() : vide complètement la liste.
    - L.index(x) : retourne l'index de la première occurrence de x
    - L.extend(iterable) : ajoute tous les éléments d'un autre iterable.
    - L.sort() → trie la liste en place. (tri ascendant).
    - L.sort(reverse=True) : tri descendant.
    - L2 = sorted(L) : retourne une nouvelle liste triée.

Les structures de bases

- **Les sets :** Un ensemble en python est semblable à une liste mais ne peut pas contenir des doublons c.à.d ne contient que des éléments distincts.

Exemple : a = {1, 2, 3} , b = {3, 4, 5}

Opérations ensemblistes :

- Union de deux ensembles : a.union(b) ou a | b # {1, 2, 3, 4, 5}
- Eléments communs : a.intersection(b) ou a & b # {3}
- Eléments présents dans le premier mais pas dans le second : a.difference(b) , a - b # {1, 2}
- Eléments dans l'un ou l'autre, pas mais pas dans les deux :
  a.symmetric_difference(b) ou a ^ b # {1, 2, 4, 5}

Tests d'inclusion

- {1, 2}.issubset(a) : test si un ensemble est inclus dans un autre # True
- a.issuperset({1}) : test si un ensemble contient un autre # True
- a.isdisjoint({10, 20}) : test si deux ensembles n'ont aucun élément commun # True

Les structures de bases

- **Les tuples :**

Les **tuples** en Python sont des **collections immuables et ordonnées**, c'est-à-dire :

- On **ne peut pas modifier** un tuple une fois qu'il est créé.
- Les éléments sont **ordonnés**, comme dans une liste.
- Les **éléments peuvent être de types différents**.

```
mon_tuple = (1, 2, 3)
tuple_vide = ()
tuple_1_element = (42,)  # virgule nécessaire !
```

Fonctions utiles :

len(t) → nombre d'éléments

t.index(x) → index de la première occurrence de x

t.count(x) → nombre d'occurrences de x

Cas d'usage fréquents : Coordonnées géographiques (latitude, longitude)

Les structures de bases

- **Les dictionnaires :**
  Le seul type associatif clé,valeur qui existe en python. Un dict est une collection non ordonnée de paires clé-valeur, très utile pour structurer des données.

- Syntaxe : dictionnaire = {cle1:val1, cle2:val2,..}
- Accéder à une valeur : dictionnaire[cle1]
- Modifier un élément : dictionnaire[cle1] = val1_bis
- Supprimer un élément : del dictionnaire[cle1] , dictionnaire.pop(cle1)
- Retourner les valeurs du dictionnaire : dictionnaire.values()
- Retourner les clés du dictionnaire : dictionnaire.keys()
- Parcourir un dictionnaire :

```
for cle, valeur in dictionnaire.items():
    print(f"{cle} : {valeur}")
```

Exercice 2 :

1. Ecrire un programme qui lit une chaine de caractère et construit un dictionnaire contenant en clé les caractères de la chaine et en valeur leur fréquence d'apparition dans la chaine

Exercice 3 :

1. Ecrire un programme qui lit un dictionnaire et qui inverse leurs clés avec leurs valeurs. C.à.d dans le nouveau dictionnaire les clés seront les valeurs de l'ancien dictionnaire et les valeurs seront les clés de l'ancien.

Exercice 4 :

1. Ecrire une fonction qui lit une liste et retourne :
   a) La liste sans doublons ordonnée dans l'ordre croissant si tous les éléments de la liste sont des entiers (Si liste contient un float, arrondir la valeur pour la traiter comme un int)
   b) La liste sans doublons ordonnée dans l'ordre décroissant si tous les éléments de la liste sont des chaînes de caractères
   c) Si la liste est composée à la fois de string et de int (ou float), la séparer en deux listes du même type et retourner deux listes en fonction de a & b.

Le formatage des variables :

Le formatage de texte en Python permet justement de **mettre en forme dynamiquement** une chaîne de caractères avec des variables

<u>Exemple</u> :

```
prenom = "Jean"
nom = "Dupont"
"Bonjour " + prenom + " " + nom + " !"
```

```
prenom = "Jean"
nom = "Dupont"
"Bonjour {} {} !".format(prenom, nom)
```

```
prenom = "Jean"
nom = "Dupont"
"Bonjour {p} {n} !".format(n=nom, p=prenom)
```

Utilisation de .format

```
prenom = "Jean"
nom = "Dupont"
f"Bonjour {prenom} {nom} !"
```

Utilisation de « f-string »

Gestion des erreurs :

- « try except » pour la gestion des exceptions & des erreurs :

```
while True:
    try:
        x = int(input("Please enter a number: "))
        break
    except:
        print("Oops!  That was no valid number.  Try again...")
```

```
Please enter a number: i
Oops!  That was no valid number.  Try again...

Please enter a number:
```

Pour afficher l'erreur :

```
while True:
    try:
        x = int(input("Please enter a number: "))
        break
    except Exception as e:
        print(f"Encountered error : {e}")
```

```
Please enter a number: i
Encountered error : invalid literal for int() with base 10: 'i'
```

Gestion des erreurs :

- « raise » déclencher une erreur

```
if True:
    raise ValueError('Aucune valeur entrée')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-3-cd29a6a8cdaa> in <module>
      1 if True:
----> 2     raise ValueError('Aucune valeur entrée')

ValueError: Aucune valeur entrée
```

Types d'erreurs possibles :

- SyntaxError : Se produit lorsqu'il y a une erreur de syntaxe dans le code, comme une parenthèse manquante ou un mot-clé mal orthographié.
- IndentationError : Se produit lorsqu'il y a des niveaux d'indentation incorrects ou incohérents dans le code, ce qui est crucial en Python pour définir les structures de bloc.
- NameError : Se produit lorsqu'un nom ou une variable n'est pas trouvé dans la portée actuelle.

Gestion des erreurs :

Types d'erreurs possibles :

- TypeError : Se produit lorsqu'une opération ou une fonction est appliquée à un objet de type inapproprié.
- ValueError : Se produit lorsqu'une fonction reçoit un argument du bon type mais avec une valeur inappropriée.
- IndexError : Se produit lorsqu'on essaie d'accéder à un index qui n'existe pas dans une séquence (par exemple, une liste, un tuple, une chaîne de caractères).
- KeyError : Se produit lorsqu'on essaie d'accéder à une clé qui n'existe pas dans un dictionnaire.
- AttributeError : Se produit lorsqu'on essaie d'accéder ou d'utiliser un attribut qui n'existe pas pour un objet donné.
- ZeroDivisionError : Se produit lors de la division par zéro.

Exercice 5 :

1. Écrivez un programme Python qui demande à l'utilisateur deux nombres et effectue la division du premier nombre par le second. Cependant, votre programme doit être capable de gérer les erreurs telles que la division par zéro ou la saisie incorrecte des nombres par l'utilisateur en affichant les bons messages contenant les valeurs entrées soient en utilisant « .format » ou les « f-string ».